Arbitrary Precision Arithmetic Library: Implementation Overview

Harikrishna S - CS24BTECH11030

May 2025

1 Introduction

This report describes the implementation of a Java-based Arbitrary Precision Arithmetic Library, designed to perform high-precision integer and floating-point arithmetic. The library 'arbritaryarithmetic' includes three core classes: `AInteger`, `AFloat`, and `ANumber`, with a focus on efficient storage and arithmetic operations.

2 Class Structure and Data Representation

2.1 ANumber

The abstract base class `ANumber` defines common properties for `AInteger` and `AFloat`. It includes:

- `num_list`: An `ArrayList<Integer>` storing digits of the number.
- A `ZeroDivisionError` exception for handling division by zero.

2.2 AInteger

The `AInteger` class handles arbitrary-precision integer arithmetic. Each digit is stored in `num_list`, where:

- Index 0 represents the units place, index 1 the tens place, and so on.
- Negative numbers are represented by storing negative digits in `num_list` (e.g., -123 is stored as [-3, -2, -1]).

2.3 AFloat

The `AFloat` class extends `ANumber` for floating-point arithmetic. It uses:

- `num_list` for digits, similar to `AInteger`.
- A `power` property, where the floating-point value is computed as $\text{num\_list} \times 10^{\text{power}}$.
- Negative numbers are represented with negative digits in `num_list`.

3 Arithmetic Operations

3.1 Addition and Subtraction

For `AInteger`:

- Digits at corresponding indices are added or subtracted.
- A separate `resolve` function handles carry-over, normalizing the result by propagating carries or borrows.

For `AFloat`:

- Operands are aligned to the same `power` by shifting digits.
- Addition or subtraction is then performed as in `AInteger`, followed by carry resolution.

3.2 Multiplication and Division

Both classes implement multiplication and division mimicking manual arithmetic:

- **Multiplication**: Each digit of one operand is multiplied by the entire second operand, with appropriate shifts (e.g., appending zeros for place values). Results are summed.
- **Division**: Uses a long division approach, iteratively subtracting multiples of the divisor from the dividend.

3.3 Helper Functions

Supporting operations include:

- `_left_shift` and `_right_shift`: Adjust the position of digits in `num_list`.
- `_negate`: Inverts the sign of the number by negating each digit.
- `_is_negative`: Checks if the number is negative based on `num_list` values.

3.4 Design diagram

4 Limitations and Potential Improvements

- The separate carry-over mechanism in `resolve` could be integrated into a single loop for efficiency.
- Multiplication performance for large numbers could be improved by memoizing intermediate results.